

Kotlin 中 Get 与 Set 方法教学及应用

Kotlin Get方法与Set方法教学讲义（零基础入门）

各位同学，今天咱们聚焦Kotlin中属性的“访问控制”核心——**Get方法与Set方法**。它们就像“数据的门卫”：Get负责“读取数据”，Set负责“修改数据”，还能在读写时加逻辑（比如“年龄不能为负数”“密码修改时记录日志”）。这节课用“生活场景+代码实操”的方式，保证小白也能轻松掌握！

一、先搞懂：什么是Get方法与Set方法？

想象一个“存钱罐”：

- **Get方法**：打开存钱罐看里面有多少钱（读取数据）；
- **Set方法**：往存钱罐里放钱（修改数据）；
- **特殊逻辑**：放钱时可以限制“单次最多放100元”，取钱时可以记录“取了多少钱”。

在Kotlin中，每个属性（`val` 或 `var`）都隐式包含Get方法和Set方法（`val` 只有Get，`var` 有Get和Set）。我们可以**自定义Get/Set的逻辑**，实现“数据读写时的额外操作”。

二、默认Get/Set：隐式存在的“默认门卫”

Kotlin中定义属性时，即使不写Get/Set，它们也会**隐式存在**，负责最基础的“读”和“写”。

Code block

```
1  class Person {  
2      // 定义一个普通属性：姓名  
3      var name: String = "小明"  
4  }  
5  
6  fun main() {
```

```

7      val person = Person()
8      // 读取name (调用隐式的get方法)
9      println(person.name) // 输出: 小明
10
11     // 修改name (调用隐式的set方法)
12     person.name = "小红"
13     println(person.name) // 输出: 小红
14 }

```

核心逻辑： `person.name` 读取时，实际调用了 `person.getName()`；修改时调用了 `person.setName("小红")` ——这就是Kotlin的“语法糖”，让代码更简洁。

三、自定义Get方法：“读取数据时的额外逻辑”

当需要在**读取属性时做额外操作**（比如“计算年龄是否成年”“格式化金额”），就可以自定义Get方法。

案例1：根据生日计算是否成年

Code block

```

1  class User(val birthday: String) {
2      // 自定义Get方法: 判断是否成年 (年龄≥18)
3      val isAdult: Boolean
4          get() {
5              // 简单模拟: 假设生日格式为“2005-01-01”, 提取年份计算
6              val birthYear = birthday.substring(0, 4).toInt()
7              val currentYear = 2025
8              return currentYear - birthYear >= 18
9          }
10 }
11
12 fun main() {
13     val user1 = User("2000-05-10")
14     println("是否成年: ${user1.isAdult}") // 输出: true
15
16     val user2 = User("2010-02-20")
17     println("是否成年: ${user2.isAdult}") // 输出: false

```

```
18 }
```

解读：`isAdult` 的Get方法不是直接返回一个固定值，而是**动态计算**“当前年份 - 出生年份是否 ≥ 18 ”，每次读取 `isAdult` 时都会执行这段逻辑。

案例2：格式化金额（保留两位小数）

Code block

```
1 class Order {
2     var price: Double = 99.9
3     // 自定义Get方法：格式化金额
4     val priceText: String
5         get() = "¥${ "%.2f".format(price)}" // 格式化为“¥99.90”
6 }
7
8 fun main() {
9     val order = Order()
10    order.price = 125.5
11    println("格式化金额: ${order.priceText}") // 输出: ¥125.50
12 }
```

四、自定义Set方法：“修改数据时的额外逻辑”

当需要在**修改属性时做校验或记录**（比如“年龄不能为负数”“密码修改时记录日志”），就可以自定义Set方法。

案例1：年龄不能为负数

Code block

```
1 class Student {
2     private var _age: Int = 0 // 真实存储年龄的“幕后字段”
3     // 自定义Set方法：校验年龄不能为负
```

```

4      var age: Int
5          get() = _age
6          set(value) {
7              if (value >= 0) {
8                  _age = value
9              } else {
10                 println("年龄不能为负数，设置失败")
11             }
12         }
13     }
14
15     fun main() {
16         val student = Student()
17         student.age = 18
18         println("年龄: ${student.age}") // 输出: 18
19
20         student.age = -5 // 触发Set方法的校验逻辑
21         println("年龄: ${student.age}") // 输出: 18 (未被修改)
22     }

```

关键： `_age` 是“幕后字段”（用 `_` 开头约定），对外暴露的 `age` 通过自定义Get/Set来控制读写逻辑。

案例2：密码修改时记录日志

Code block

```

1  class Account {
2      private var _password: String = "123456"
3      // 自定义Set方法: 记录密码修改日志
4      var password: String
5          get() = _password
6          set(newPwd) {
7              println("密码从'$_password'修改为'$newPwd'")
8              _password = newPwd
9          }
10 }
11
12 fun main() {
13     val account = Account()
14     account.password = "abc123" // 触发Set方法的日志逻辑

```

```
15      // 输出：密码从 '123456' 修改为 'abc123'
16  }
```

五、Get/Set在Android开发中的实战场景

场景1：ViewModel中封装数据（避免直接修改）

Code block

```
1  class UserViewModel : ViewModel() {
2      private val _userList = mutableListOf<User>() // 幕后字段（可变）
3      // 对外暴露不可变的列表（只能读，不能改）
4      val userList: List<User>
5          get() = _userList
6
7      fun addUser(user: User) {
8          _userList.add(user)
9          // 通知UI刷新（省略LiveData逻辑）
10     }
11 }
```

好处：外部只能通过 `userList` 读取数据，无法直接修改列表（比如调用 `userList.add()`），保证数据安全。

场景2：自定义View的属性（支持XML配置）

Code block

```
1  class CustomButton @JvmOverloads constructor(
2      context: Context,
3      attrs: AttributeSet? = null,
4      defStyleAttr: Int = 0
5  ) : Button(context, attrs, defStyleAttr) {
6
7      // 自定义属性：按钮圆角大小
```

```
8      var cornerRadius: Float = 0f
9      set(value) {
10          field = value
11          updateBackground() // 修改属性后刷新背景
12      }
13
14      private fun updateBackground() {
15          // 根据cornerRadius设置圆角背景（省略具体实现）
16      }
17  }
```

作用：在XML中配置 `app:cornerRadius="10dp"` 时，会触发Set方法，自动刷新按钮的圆角效果。

总结与作业

核心知识点

- Kotlin的每个属性都隐式包含Get/Set方法，`val` 只有Get，`var` 有Get和Set；
- 自定义Get方法：在读取属性时执行额外逻辑（如动态计算、格式化）；
- 自定义Set方法：在修改属性时执行校验、日志等逻辑，通过“幕后字段”（如 `_age`）存储真实值；
- 实战价值：封装数据、控制读写权限、实现UI属性的联动刷新。

课后作业

定义一个 `BankAccount` 类，包含以下逻辑：

1. 属性 `balance` （余额），默认0；
2. 自定义Set方法：存款时（`balance` 增加）必须 ≥ 0 ，取款时（`balance` 减少）不能让余额为负；
3. 自定义Get方法：返回格式化的余额字符串（如“当前余额：¥100.00”）；

4. 测试存款（+100）、取款（-50）、尝试取款150（余额不足）的场景。

试着自己写代码实现，有问题随时问哦！

（注：文档部分内容可能由 AI 生成）